

Constrained Generation of Theme-Based Hindi Dohas Using Indic Language Models

Divyanshu Jain Kavan Gandhi Nikhil Patidar Abhinav Jha

2025201059

2025201078

2025201081

2024201008

Abstract

This project investigates the problem of generating Hindi dohas under strict poetic constraints. A doha is a two-line Hindi poem that follows a fixed matra-based metrical structure and conveys a central theme or lesson. The objective of this work is to generate such dohas automatically given a user-defined theme and context while ensuring structural correctness and semantic coherence.

1 Introduction

Large Language models are particularly effective at producing fluent and semantically coherent text in free form settings. However, generating text that must strictly conform to predefined structural or formal constraints remains a challenging problem. Poetry generation is a prominent example of such constrained text generation, where outputs must satisfy rules related to meter, line length, rhyme, and overall structure. Limited work has been done especially in poetry generation for Indic Languages.

This project addresses the identified gap by focusing on the generation of Hindi dohas, a structurally rigid poetic form. Given a user-provided theme and context, our objective is to generate grammatically correct and semantically coherent dohas that strictly adhere to the necessary constraints. To achieve this, we curate a specialized dataset of Hindi dohas and train an Indic language model on this data. The generated outputs are evaluated for structural and matra correctness.

All code, datasets, and model checkpoints are publicly available at: <https://github.com/KGan31/Doha-Gen>

2 Relevant Work

The doha is a traditional form of Hindi poetry widely used by saint-poets such as Kabir and Rahim, and is characterized by a highly constrained prosodic structure. A doha consists of exactly two lines, each line being divided into two halves, resulting in four rhythmic phases. Unlike English poetry, which is often based on syllable stress, Hindi poetry follows a matra-based prosodic system, where a matra represents a unit of syllabic duration. In valid doha, the first half of each line contains 13 matras, while the second half contains 11 matras, and both lines typically end in a rhyme, contributing to its musical and lyrical quality. Due to these strict formal constraints, generating structurally valid dohas poses a significant challenge for modern language models, making them a suitable testbed for constrained text generation in Indic languages.

Chandomitra ([Jagadeeshan et al., 2025](#)) presents a structured dataset for translating English input into Sanskrit poetry that strictly adheres to the Anustubh meter, a classical Sanskrit metrical form. The work emphasizes the importance of explicitly modeling prosodic constraints to ensure metrical correctness in generated poetry. It demonstrates that unconstrained language models are insufficient for classical Indian poetic forms and motivates the use of hard constraints when dealing with matra- or syllable-governed meters.

The Mechanical Bard ([Agnew et al., 2023](#)) explores constrained poetry generation in English by generating Shakespearean sonnets using a pre-trained GPT-2 language model. The authors enforce a combination of hard constraints, such as syllable counts and rhyme schemes, and soft constraints, such as stylistic and poetic coherence, during the generation

process.

Our proposed work draws inspiration from both of these studies. Similar to Chandomitra, we emphasize the necessity of prosodic correctness for Indic poetic forms, and following The Mechanical Bard, we adopt a constraint-driven generation paradigm built on top of large language models. However, our focus is on theme-conditioned generation of Hindi dohas in Devanagari script, a relatively underexplored poetic form governed by matra-based constraints.

3 Proposed Methodology and Architecture

The proposed system generates theme-based Hindi dohas in Devanagari script while adhering to the formal constraints of the doha poetic form. Our pipeline follows a two-stage training strategy followed by constrained inference:

1. **Stage 1: Continued Pretraining (Domain Adaptation):** The base model is further pretrained on a corpus of $\approx 40,000$ Hindi poems using a denoising objective, adapting it from general web-Hindi to the domain of Hindi poetry.
2. **Stage 2: Supervised Fine-tuning (Task Learning):** The domain-adapted model is fine-tuned on $\approx 8,000$ annotated (theme + context, doha) pairs to learn conditional doha generation.
3. **Constrained Inference:** Beam search generates multiple candidates, which are reranked by a rule-based matra validator enforcing the 13–11–13–11 prosodic pattern.

Figures 1 and 2 detail the architecture of each training stage.

3.1 Data Collection and Preprocessing

Our data pipeline collects two complementary corpora: a large set of general Hindi poems for domain adaptation and a smaller set of annotated dohas for supervised task learning.

Hindi poetry corpus (Stage 1). We use approximately 40,000 Hindi poems sourced from KavitaKosh (KavitaKosh), spanning genres including ghazals, nazms, and free-verse poetry. Each record contains the poet name, poem title, and poem content in Devanagari.

This corpus serves to teach the model general Hindi poetic style—figurative language, rhyme, couplet structure, and literary vocabulary.

Doha corpus (Stage 2). We scrape Hindi dohas from KavitaKosh into a structured CSV with columns: **Author**, **Doha**, **Theme**, **Meaning**, and **Context**. A detailed analysis of dataset statistics and quality is presented in Section 5.1.

3.2 Base Model: IndicBART

We use IndicBART (Dabre et al., 2022) as our base model, a multilingual sequence-to-sequence model with approximately 244M parameters based on the mBART architecture. IndicBART was pretrained on 11 Indian languages using a denoising objective (span masking with sentence permutation) over the IndicCorp corpus. We select IndicBART for three reasons:

1. **Tokenizer efficiency:** Its SentencePiece vocabulary is optimized for Devanagari, producing fewer subword tokens per Hindi word than multilingual models like mT5 or mBART-cc25.
2. **Hindi data proportion:** Hindi constitutes a significantly larger share of IndicBART’s pretraining data than in massively multilingual models where it may be only 1–2%.

The model uses a standard Transformer encoder-decoder with 6 layers each, hidden dimension 1024, 16 attention heads, and a shared vocabulary of 64,000 tokens. Language-specific tokens (e.g., <2hi>) are appended to signal the target language.

3.3 Stage 1: Continued Pretraining (Domain Adaptation)

Although IndicBART understands Hindi, it was pretrained on general web text (news, Wikipedia). It has limited exposure to poetic language figurative expressions, metrical rhythm, rhyming couplets, and literary vocabulary. Stage 1 bridges this gap by continuing the denoising pretraining on our 40,000 Hindi poem corpus.

Denoising objective. We apply the same denoising objective used in IndicBART’s original pretraining, following the mBART formu-

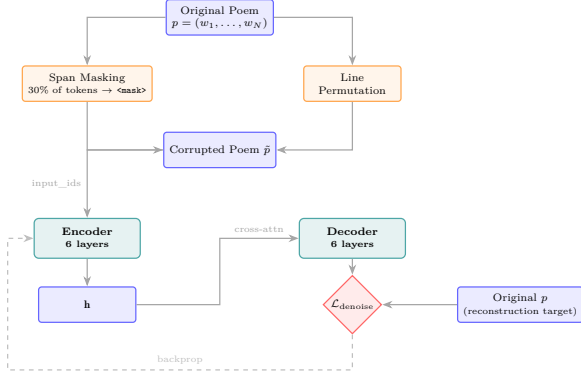


Figure 1: Stage 1: Continued pretraining via denoising. The original poem is corrupted by span masking (30%) and line permutation. The encoder-decoder is trained to reconstruct the original poem from the corrupted version.

lation (Liu et al., 2020). Each poem is corrupted using two noise functions applied sequentially:

1. **Span masking (30%):** Contiguous spans of tokens are replaced with a single `<mask>` token. Approximately 30% of tokens are masked, with a mean span length of 3.
2. **Sentence permutation:** The lines of the poem are randomly shuffled.

The model is trained to reconstruct the *original, uncorrupted* poem from the corrupted input. Given a poem $p = (w_1, \dots, w_N)$ and its corrupted version $\tilde{p}$, the loss is:

$$\mathcal{L}_{\text{denoise}} = - \sum_{t=1}^N \log P_{\theta}(w_t | w_{<t}, \text{Enc}(\tilde{p})) \quad (1)$$

Concrete example. Given an original poem:

नदी किनारे बैठा मन, चुपचाप समय को देखे।
लहरों की धीमी सरगम में, जीवन के सुर सीखे।
चलते जल की चाल में, बदलती दुनिया झलके।
क्षणभंगुर इस जीवन में, हर पल को मन परखे।

The corrupted encoder input (after line permutation + span masking) might be:

चलते जल की चाल में, बदलती दुनिया झलके।
नदी किनारे बैठा मन, `<mask>` समय को देखे।
क्षणभंगुर इस जीवन में, हर `<mask>` को मन परखे।
लहरों की धीमी `<mask>` में, जीवन के सुर सीखे।

The decoder is trained to produce the original four-line poem in its correct order with all words restored.

What Stage 1 teaches.

- **Poetic structure:** Couplet forms, line-level organization, and consistent line lengths.
- **Figurative language:** Metaphors, similes, and allegorical expressions common across all Hindi poetry.
- **Rhyme patterns:** Recurring phonetic patterns at line endings.
- **Literary vocabulary:** Words and phrases that occur in poetry but rarely in news or Wikipedia text.

This represents a *domain shift* from general web-Hindi to Hindi poetry, which Stage 2 further narrows to the specific doha form.

Validation. Stage 1 is unsupervised, so validation is indirect. We hold out 5% of poems and monitor:

- **Perplexity** on the held-out set: should decrease relative to the base model, indicating improved fluency on poetic text.
- **Qualitative fill-in-the-blank:** we mask known words in poetic text and check whether the model’s predictions are contextually appropriate.

3.4 Stage 2: Supervised Fine-tuning with Differentiable Prosody Control

Stage 2 starts from the Stage 1 checkpoint (poetry-adapted IndicBART) and learns theme-conditioned doha generation with an additional differentiable metrical objective. The key idea is to keep standard teacher-forced seq2seq learning, but add a soft prosody branch that computes expected matras directly from token probability distributions.

Task formulation. For each training sample i , the encoder input is:

$$x_i = \text{विषय: } \langle \text{theme} \rangle \mid \text{संदर्भ: } \langle \text{context} \rangle \langle 2\text{hi} \rangle$$

and the decoder target is:

$$y_i = (y_{i,1}, y_{i,2}, \dots, y_{i,T})$$

where y_i is the gold doha text.

Model backbone. We retain the IndicBART encoder-decoder architecture from Stage 1:

1. Encoder (6 Transformer layers) processes x_i .

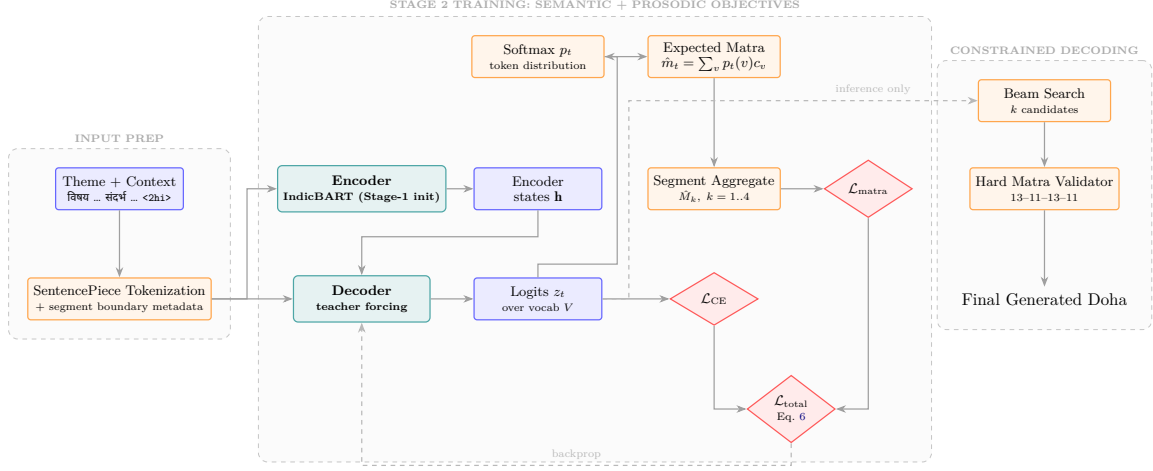


Figure 2: Stage 2 architecture with differentiable expected-matra supervision during training and hard matra-constrained decoding at inference.

2. Decoder (6 Transformer layers) predicts next-token logits under teacher forcing.
3. Shared SentencePiece vocabulary and Hindi BOS control token <2hi>.

Cross-entropy objective (semantic fluency).

$$\mathcal{L}_{\text{CE}} = - \sum_{t=1}^T \log P_{\theta}(y_{i,t} \mid y_{i,<t}, \text{Enc}(x_i)) \quad (2)$$

Soft matra expectation branch. Let $z_t \in R^{|V|}$ be decoder logits at step t , and

$$p_t = \text{softmax}(z_t), \quad p_t(v) = P_{\theta}(v \mid y_{<t}, x_i).$$

Define a fixed matra-cost vector $c \in R^{|V|}$, where $c_v = m(v)$ is the matra value assigned to token v . Expected matra contribution at step t :

$$\hat{m}_t = \sum_{v \in V} p_t(v) c_v = p_t^{\top} c \quad (3)$$

Segment-wise doha constraints. A doha has 4 metrical segments with targets

$$M^{\star} = [13, 11, 13, 11].$$

Let S_k denote token positions belonging to segment k ($k \in \{1, 2, 3, 4\}$). The expected segment matra is:

$$\hat{M}_k = \sum_{t \in S_k} \hat{m}_t \quad (4)$$

and the prosody loss is:

$$\mathcal{L}_{\text{matra}} = \sum_{k=1}^4 \text{SmoothL1}(\hat{M}_k, M_k^{\star}) \quad (5)$$

Joint Stage 2 objective.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{matra}} \quad (6)$$

where λ controls semantic-prosodic tradeoff.

Inference-time constrained decoding. During generation, we apply hard enforcement:

1. Beam search produces k candidate sequences.
 2. A matra validator enforces 13–11–13–11 at segment level.
 3. The best valid candidate is selected by beam score (or reranked composite score).
- This gives a hybrid setup: *soft differentiable control during training + hard constraint satisfaction during decoding.*

4 Evaluation Metrics

To evaluate the quality of the generated dohas, we employ a combination of structural, and novelty.

4.1 Matra Accuracy Score (MAS)

Hindi prosody is based on *matras* units of syllabic duration rather than stress. Each character in Devanagari contributes a fixed number of matras: short vowel sounds (*laghu*) contribute 1 matra, and long vowel sounds (*guru*) contribute 2 matras. Consonants in conjunct clusters and half-characters follow additional rules. For example:

- **क** (ka, short) $\rightarrow$ 1 matra
- **का** (kaa, long) $\rightarrow$ 2 matras

- क (half consonant) $\rightarrow$ 0 matras (merges with next character)

A valid doha has four *charans* (quarter-lines) with the pattern **13 + 11 | 13 + 11**, totalling 48 matras.

4.1.1 How the Matra Counter Works

The matra counting function takes a doha string as input and produces per-charan matra counts as output. The processing pipeline is as follows:

1. **Charan Splitting:** The doha is split into two lines using the *daari* symbols (| or ||). Each line is then split into two charans at the comma (,) separator, yielding four charans C_1, C_2, C_3, C_4 .
2. **Character-level Matra Assignment:** Each Devanagari character is classified by its Unicode category vowel matras, independent vowels, consonants, half-forms, anusvara, visarga, nukta and assigned its corresponding matra weight per classical Hindi prosody rules.
3. **Charan Matra Count:** The matra weights are summed over each charan to yield counts m_1, m_2, m_3, m_4 .
4. **Validity Check:** The doha is structurally valid if and only if $m_1 = 13, m_2 = 11, m_3 = 13, m_4 = 11$.

4.1.2 Matra Accuracy Score Formula

For a generated doha with charan matra counts m_1, m_2, m_3, m_4 , MAS is the total absolute deviation from the ideal pattern:

$$\text{MAS} = |m_1 - 13| + |m_2 - 11| + |m_3 - 13| + |m_4 - 11| \quad (7)$$

A perfectly valid doha scores $\text{MAS} = 0$. Unlike a binary valid/invalid flag, MAS is a continuous metric it quantifies *how far* each output is from correctness, making it suitable for comparing models that are all structurally imperfect to varying degrees.

4.1.3 Role of MAS in the Pipeline

MAS serves two distinct roles. As an **evaluation metric**, it is computed over all generated outputs to compare models. A model with lower average MAS is producing more structurally correct dohas.

4.1.4 Evaluation on Scraped Dataset

Running the matra counter on our scraped dataset of 7,889 dohas revealed important insights about real-world doha text quality, summarized in Table 1.

Table 1: Matra Counter Results on Scraped Dataset

Metric	Value
Total dohas evaluated	7,889
Dohas with all 4 charans parsed	6326 (80.2%)
Dohas collapsed to 2 charans	1243 (15.8%)
Flagged valid (48 total matras)	5257 (66.6%)

A key limitation surfaced: 1243 dohas lacked a comma separator, causing the parser to treat each full line as a single unit and produce counts of the form $\langle 24, 24, 0, 0 \rangle$. Such dohas are marked valid if the total reaches 48, but this does not verify the 13+11 split within each line. The 66.6% validity figure is therefore an overestimate of true structural correctness. Fixing the charan-splitting logic to handle alternative separators is a near-term task.

Out of the full dataset of **7,889 dohas**, **7,405 dohas (93.9%)** fall within a ± 2 matra tolerance of the ideal **48-matra** target, with only **484 dohas (6.1%)** deviating beyond this range — confirming that the scraped corpus is of high prosodic quality.

4.2 BLEU Score

BLEU (Bilingual Evaluation Understudy) measures the n-gram overlap between a generated text and one or more reference texts. For a generated sequence $\hat{y}$ and a set of reference sequences $\{y_1, y_2, \dots, y_m\}$, BLEU- n is computed as:

$$\text{BLEU-}n = BP \cdot \exp \left(\sum_{k=1}^n w_k \log p_k \right) \quad (8)$$

where p_k is the modified k -gram precision, $w_k = \frac{1}{n}$ is the uniform weight for each k -gram order, and BP is the brevity penalty to discourage overly short generations:

$$BP = \begin{cases} 1 & \text{if } |\hat{y}| > |y| \\ e^{1-|y|/|\hat{y}|} & \text{if } |\hat{y}| \leq |y| \end{cases} \quad (9)$$

Higher BLEU scores indicate greater n-gram overlap with reference dohas.

4.3 Novelty Score (NS)

Novelty is measured using Self-BLEU to quantify diversity among generated dohas. For a set $\{D_1, D_2, \dots, D_n\}$:

$$\text{NS} = 1 - \frac{1}{n} \sum_{i=1}^n \text{BLEU}(D_i, \{D_j\}_{j \neq i}) \quad (10)$$

Higher values indicate greater diversity. This guards against degenerate models that repeatedly produce the same output or closely paraphrase training examples.

5 Progress

5.1 Dataset Creation

Data Collection from KavitaKosh. The primary corpus was collected from *KavitaKosh* ([KavitaKosh](#)), one of the most comprehensive online repositories of Hindi poetry. KavitaKosh hosts thousands of dohas attributed to classical and contemporary poets spanning multiple centuries and literary traditions.

We developed a Python scraper using `BeautifulSoup` and `Requests` to systematically traverse doha-specific sections of the website. Each entry was stored with its attributed **author name** and **source URL** for traceability and post-hoc verification. The initial crawl yielded a raw collection of over 8,500 candidate doha entries.

Data Cleaning and Preprocessing. Raw scraped data required multi-stage cleaning before use in model training. The following preprocessing steps were applied in sequence:

1. **Deduplication:** Exact-string matching after Unicode NFC normalisation removed duplicate entries arising from poems appearing in multiple KavitaKosh categories.
2. **Structural filtering:** Entries not conforming to the two-line doha structure—extra verses or incomplete fragments—were discarded.
3. **Script sanitisation:** HTML parsing artefacts, stray ASCII punctuation, and corrupt Devanagari sequences were removed via regex-based rules.

4. **Unicode normalisation:** NFC form was applied uniformly to ensure consistent encoding of Devanagari characters, mātrās, and combining marks.

After cleaning, the final dataset comprised **7,889 high-quality dohas** distributed across **67 unique authors**.

Semantic Annotation via Gemini API.

Each doha was enriched with three Gemini-generated fields: (1) **Theme** : a concise thematic label (e.g., *Bhakti*, *Prem*, *Niti*), resulting in 104 distinct thematic categories; (2) **Context** : a brief one-line situational backdrop of the doha; and (3) **Meaning** : a full plain-language explanation in simple modern Hindi. These fields are stored alongside the original Devanagari doha text, making the dataset directly usable for sequence-to-sequence fine-tuning.

Dataset Schema Table 2 summarises the six fields present in the final UTF-8 CSV dataset. Three fields *Theme*, *Context*, and *Meaning* were generated by the Gemini API, while the remaining three were obtained directly through scraping.

Table 2: Schema of the KavitaKosh Doha Dataset

Field	Source	Description
Author	Scraped	Poet name (Devanagari script)
Doha	Scraped	Full two-line doha in Devanagari
URL	Scraped	Canonical KavitaKosh source URL
Theme	Gemini API	Thematic label assigned by Gemini
Meaning	Gemini API	Full explanation of the doha in plain language
Context	Gemini API	Brief one-line situational backdrop of the doha

Dataset Statistics & Analysis. Table 3 lists the twelve most frequent thematic categories. The corpus is dominated by *Prem* (Love), *Samajik* (Social), *Bhakti* (Devotion), and *Niti* (Ethics), together accounting for over 34% of all entries. The five most prolific poets : Rasleen (911), Deenanath Sum-

itr (798), Bharatendu Mishra (698), Ranjana Verma (643), and Dharanidas (510) account for approximately 45% of the corpus.

Table 3: Top 12 Thematic Categories

Rank	Theme	Translation	Count
1	Prem	Love	809
2	Samajik	Social	699
3	Bhakti	Devotion	641
4	Niti	Ethics	605
5	Virah	Longing	352
6	Jivan	Life	339
7	Karun	Compassion	327
8	Shringar	Aesthetics	325
9	Saundarya	Beauty	320
10	Rajaniti	Politics	309
11	Prakrti	Nature	216
12	Riti	Tradition	204
–	Others	92 remaining themes	2,743
Total			7,889

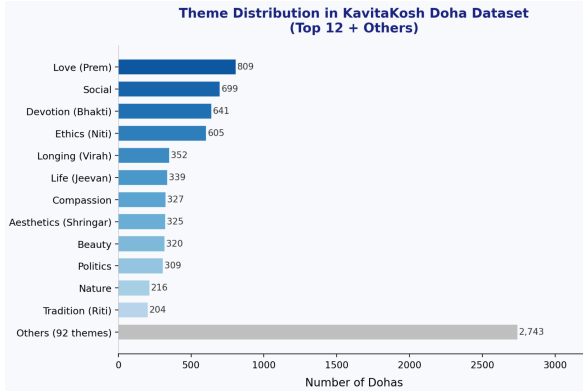


Figure 3: Theme frequency distribution: top 12 themes + Others (92 themes, 2,743 dohas).

Dataset Quality Considerations.

Since theme labels are generated by Gemini rather than manually curated, minor inconsistencies may exist (e.g., a doha assigned to both *Prem* and *Śrīṅār* on different runs). The Gemini-generated *Theme*, *Context*, and *Meaning* fields represent AI-generated approximations of classical scholarly interpretation and may occasionally deviate from established philological readings. Finally, the author-frequency distribution is skewed: five poets contribute nearly half of all entries, which may bias a fine-tuned model towards their stylistic characteristics. Stratified sampling or data augmentation could mitigate this imbalance during training.

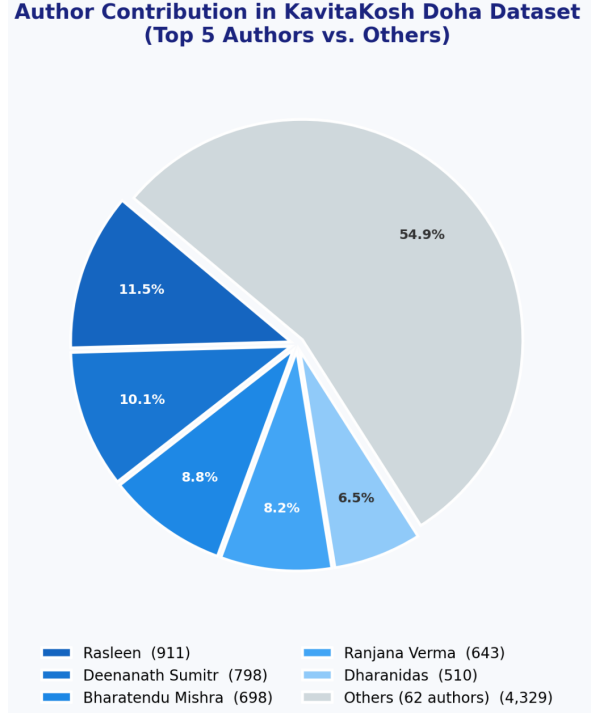


Figure 4: Author contribution: top 5 poets vs. 62 other contributors.

5.2 Baselines

5.2.1 N-Gram Model Analysis

An N-gram language model was implemented as the simplest statistical baseline for doha generation. The model was trained on 7,100 dohas (90% split) and evaluated on 789 held-out dohas. Character-level tokenization was used as the primary mode, producing a vocabulary of 134 unique tokens from 588,129 total characters.

Generation Results. Despite low perplexity, the generated outputs had no resemblance to valid dohas.

Seed: कबीर स

Output: कबीर समा प्रहे मीर साधु जान मृतक
समें की करे सुख की कहा के ना रों मुख में सुनी
करेम कमल रियारिक बाल हो जग न की

BLEU & Perplexity Analysis:

Character-level BLEU 0.825 are inflated by the small 134-token vocabulary and do not reflect generation quality. Word-level trigram results exposed the true weakness training PPL of 4.08 collapsed to validation PPL of 2864.11, revealing severe sparsity across 25,417 unique tokens.

Failure Analysis: The N-gram model fails on all three fronts: no semantic understanding of theme or poetic intent, no mechanism to track or enforce the matra structure, and data sparsity at the word level where unseen classical Hindi combinations dominate at inference time. Therefore, it serves purely as a statistical lower bound in our baseline comparison.

5.2.2 LSTM Model Analysis

A character-level Long Short-Term Memory (LSTM) network was trained as the second baseline. Unlike N-gram models, the LSTM learns distributed representations of characters and maintains a gated hidden state across time steps, giving it the theoretical ability to capture longer-range dependencies. The model architecture consisted of a character embedding layer (64 dimensions) followed by a 2-layer stacked LSTM (256 hidden units per layer) with dropout (0.5) and a linear output projection, totalling 899,078 trainable parameters.

Generation Results. The model overfitted the entire doha dataset in just one epoch

Seed: मोर पच्छ

Output (Epoch 10): मोर पच्छान ॥ वै
लाजै के प्रान कै, प्रभु करै कहि मुख दिननाइ
सुमान ॥ राम रूप भी शाल स

The model produces fragments that superficially resemble doha formatting but contain non-words (पच्छाय, च्यामाता), mixed modern and archaic Hindi, and no consistent thematic or semantic thread.

Outputs superficially mimic doha formatting but contain non-words, mixed registers, and no semantic coherence. High character-level BLEU scores (BLEU-4: 0.9427) are misleading they reflect the small 134-character Devanagari vocabulary, not generation quality.

Failure Analysis. The LSTM overfits due to insufficient data (899K parameters vs. $\approx$ 588K training characters), memorising rather than generalising.

5.2.3 GPT-2 (Hindi) Model Analysis

The fourth baseline used `surajp/gpt2-hindi` (Patil, 2020), a Hindi-adapted GPT-2 model

with 81.9M parameters pre-trained on Hindi web text. Unlike the encoder-decoder architectures of IndicBART and mT5, GPT-2 is a causal (decoder-only) language model. Two evaluation regimes were conducted: a zero-shot setting using prompt templates, and a fine-tuned setting using the same (theme, context $\rightarrow$ doha) pairs as other baselines. For fine-tuning, each training example was formatted as a single autoregressive sequence:

विषय: [theme]
संदर्भ: [context]
दोहा: [doha text] <|endoftext|>

Without fine-tuning, outputs were entirely off-distribution modern colloquial prose with no verse structure or matra awareness. After fine-tuning, the model produced full Hindi words with vowel matras intact and occasional danda markers (॥), representing a qualitative step above the LSTM and IndicBART baselines.

Generation Results. The fine-tuned model’s outputs represent a qualitative step above the LSTM and IndicBART baselines. The model correctly generates full Hindi words with vowel matras intact, occasionally inserts danda markers (॥), and shows weak thematic relevance to the input prompt.

At epoch 6, the model produced:

Output: बार बार है अँगुली, छॉटे नीके तूफान।
सूरज उग आया सोहता, रिपु अकेला जान ॥

However, two characteristic failure modes persisted: code-switching (English text mid-generation due to mixed-script pretraining data), and structurally incoherent continuations that drift into story-like prose after the opening lines. No generated sample satisfied the matra constraint. With a best validation perplexity of 12.27, GPT-2 produces more coherent Hindi words than IndicBART due to subword tokenization, but remains the weakest baseline in terms of structural validity.

5.2.4 IndicBART Model Analysis

IndicBART with 440M parameters, it is the largest model evaluated in our baseline study. Generation uses a language tag <2hi> appended to the input to direct the model towards Hindi output. The model was fine-tuned

Table 4: Baseline Model Comparison Perplexity and BLEU Scores

Model	Val PPL	BLEU-4
N-gram (4-gram KN)	9.57	0.825
LSTM (char-level)	7.07	0.9427
GPT-2 (Hindi)	12.27	0.8177
IndicBART	57.73	0.5358
mT5-base (fine-tuned)	14.47	0.1100

on 7,100 (theme + context $\rightarrow$ doha) input-output pairs for 6 epochs with a linear warmup schedule, using the input format:

विषय: theme , संदर्भ: context

Training Dynamics. Both training and validation loss decreased consistently across all 6 epochs, with validation perplexity improving from 160.15 at epoch 1 to 61.06 at epoch 6 the best checkpoint saved. A validation perplexity of 61.06 was achieved which is quite high indicating IndicBart struggled with Doha Generation despite high parameter count.

Generation Results.

Input: विषय: विरह, संदर्भ: विरह की अग्नि
Output (Epoch 3): <s><2hi>वरह क
 अगन क कय, कय न कय कय। कय दन क कछ
 क, कछ न कछ सखय ॥सहत हय कछ ...

Several problems are immediately visible. The output retains the raw special tokens (<s>, <2hi>) indicating the model never learned to suppress them. More critically, all vowel matras are stripped from words विरह becomes वरह, अग्नि becomes अगन a symptom of the IndicBART tokenizer’s subword vocabulary not aligning well with the classical Hindi script patterns in our dataset. The output does weakly reflect the theme (repetition of root words related to विरह and अग्नि) but produces no valid Hindi words, no coherent meaning, no two-line structure, and no matra validity whatsoever.

Failure Analysis. Despite 440M parameters, IndicBART’s pre-training on modern colloquial web text creates a fundamental mismatch with classical Braj Bhasha vocabulary, and 7,100 fine-tuning examples are insufficient to bridge this gap. Its SentencePiece tokenizer further strips vowel matras from classical Devanagari words a structural

incompatibility that explains why the smaller mT5-base, with a more faithful multilingual tokenizer, outperformed it decisively.

5.2.5 mT5 Model Analysis

mT5 (Xue et al., 2021) is a multilingual encoder-decoder transformer pretrained on the mC4 corpus covering 101 languages including Hindi, making it naturally suited to conditional generation tasks. We experimented with **mT5-small** (300M parameters) and **mT5-base** (580M parameters), both fine-tuned using LoRA to make training feasible on limited GPU resources.

mT5-small : After fine-tuning on our doha dataset, the output showed marginal improvement the model began producing Devanagari characters but the result remained structurally and semantically invalid:

Output: <extra_id_0> , न । ॥
 <extra_id_1> र ही ते ई

The output contains punctuation marks (., !, ॥) and isolated Hindi characters, suggesting the model learned surface-level token patterns punctuation placement, script but failed to produce coherent words or any recognisable poetic structure.

mT5-base : Fine-tuning mT5-base on our dataset for 5 epochs produced the most coherent output observed across all baseline models:

Output: कल्याण की नई भक्ति, सभी के प्रति
 समर्पण, प्रकार माँवति का. उसके प्रति सम-
 र्पण तत्त्व को ॥

The output contains full Hindi words, thematic vocabulary related to devotion (भक्ति, समर्पण, कल्याण), and ends with the doha terminator ॥ showing the model has learned the doha format to some extent. However, the output is a single run-on line rather than two structured lines, the matra counts are not valid, and parts of the text are semantically garbled (माँवति का is not a standard Hindi phrase). This represents the best unconstrained baseline output but still falls far short of a valid doha.

Quantitative Evaluation of mT5-base (Fine-tuned) We computed perplexity and BLEU score for the fine-tuned mT5-base output.

A perplexity of 14.47 indicates the model is reasonably confident in its own outputs it is not generating randomly. The BLEU score of 0.11, while low in absolute terms, is non-trivial for this task: the output shares some n-gram overlap with reference dohas on the same theme, confirming that fine-tuning did impart some stylistic learning.

5.3 Objectives In Progress

Scraping a large-scale corpus of general Hindi poems ($\approx 40,000$ poems) from KavitaKosh ([KavitaKosh](#)) for Stage 1 domain adaptation pretraining. Web scraping at this scale is time-intensive due to rate limiting and per-page crawling across poet- and genre-specific index pages.

5.4 Objectives Scheduled

1. **Architecture Implementation:** Implement the full two-stage pipeline using IndicBART as the base model, including the matra supervision branch and constrained decoding module.
2. **Stage 1 Training (Domain Adaptation):** Pretrain the model on the scraped Hindi poems corpus ($\approx 40,000$) using the denoising objective to adapt IndicBART from general web-Hindi to Hindi poetic style.
3. **Stage 2 Training (Task Fine-tuning):** Fine-tune the domain-adapted model on the annotated doha dataset ($\approx 8,000$ pairs) for theme-conditioned doha generation with prosody supervision.
4. **Evaluation and Model Selection:** Evaluate generated dohas on MAS, and NS metrics. If results fall below acceptable thresholds, switch the base model from IndicBART to mT5-base, which demonstrated stronger baseline performance in our experiments.

6 Conclusion

This report presents the first phase of our system for constrained Hindi doha genera-

tion. We have assembled a high-quality annotated dataset of 7,889 dohas, implemented a robust matra evaluation framework, and benchmarked five baseline models. Our experiments demonstrate that standard generative models regardless of scale cannot satisfy the 13+11+13+11 prosodic constraint without explicit structural supervision. These findings motivate our proposed two-stage pipeline combining domain-adapted IndicBART with a differentiable matra objective and hard-constraint decoding. Completing and evaluating this pipeline constitutes the primary goal of the remaining project timeline.

References

- Edwin Agnew, Michelle Qiu, Lily Zhu, Sam Wiseman, and Cynthia Rudin. 2023. [The mechanical bard: An interpretable machine learning approach to Shakespearean sonnet generation](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1627–1638, Toronto, Canada. Association for Computational Linguistics.
- Raj Dabre, Himani Shrotriya, Anoop Kunchukuttan, Ratish Puduppully, Mitesh Khapra, and Pratyush Kumar. 2022. [IndicBART: A pre-trained model for indic natural language generation](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1849–1863, Dublin, Ireland. Association for Computational Linguistics.
- Zhiyuan Hu, Chumin Liu, Yue Feng, Anh Tuan Luu, and Bryan Hooi. 2023. [Poetrydiffusion: Towards joint semantic and metrical manipulation in poetry generation](#). *Preprint*, arXiv:2306.08456.
- Manoj Balaji Jagadeeshan, Samarth Bhatia, Pre-tam Ray, Harshul Surana, P AkhilRajeev, Priya Mishra, Annarao Kulkarni, Ganesh Ramakrishnan, AP Prathosh, and Pawan Goyal. 2025. [Chandomitra: Towards generating structured sanskrit poetry from natural language inputs](#).
- KavitaKosh. [Kavitakosh: A repository of hindi poetry](#).
- Piji Li, Haisong Zhang, Xiaojiang Liu, and Shuming Shi. 2020. [Rigid formats controlled text generation](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 742–751, Online. Association for Computational Linguistics.
- Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike

Lewis, and Luke Zettlemoyer. 2020. [Multilingual denoising pre-training for neural machine translation](#). *Preprint*, arXiv:2001.08210.

Suraj Patil. 2020. [Gpt-2 hindi](#).

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. [mt5: A massively multilingual pre-trained text-to-text transformer](#). *Preprint*, arXiv:2010.11934.